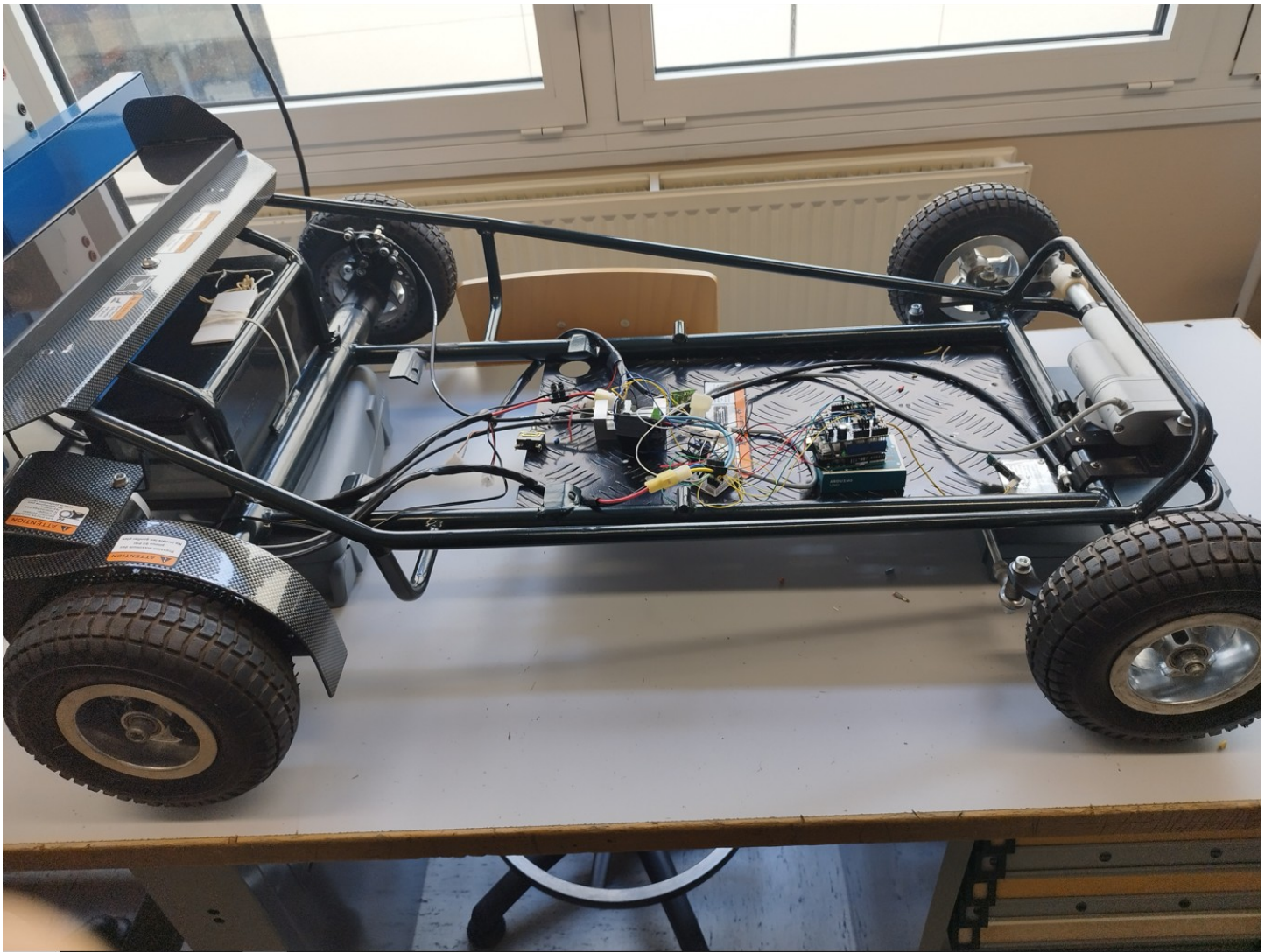


Baccalauréat technologique série STI2D

Projet de Terminal SIN 72 heures

Commande d'un kart à distance

Immaculée Conception Laval



Debonne Kylian

16 décembre 2022 – 22 mai 2023

Mr DAVID

Sommaire

1. Introduction

Cahier des charges et tâches personnelles.....	p1
Solutions retenues.....	p2
Diagramme sysml de définition de blocs.....	p3

2. Caractérisation

Schéma d'ensemble.....	p4
Caractéristiques du vérin électrique.....	p4
Caractéristiques du moteur du kart.....	p5

3. Mise en œuvre

Diagramme sysml de blocs internes.....	p5
Schéma électrique.....	p6
Programme pour le vérin électrique et le moteur du kart.....	p6
Trame des données envoyés par bluetooth.....	p9
Fonctionnement du logiciel pour la tablette.....	p10

4. Problèmes rencontrés

Direction imprécise.....	p12
Connexion bluetooth.....	p12

5. Intégration.....p14

6. Conformité au cahier des charges.....p15

7. Conclusion personnelle.....p15

Introduction

Cahier des charges et tâches personnelles

Pour finaliser notre année et terminal STI2D, nous avons un projet de plus de 72 heures à réaliser. Il s'agit d'un projet de groupe, mon groupe est constitué de : Akram Lakhil (SIN), Théo Allaire (ITEC) et Adrien Chauvel (ITEC) . Voici un extrait du diagramme des exigences et du diagramme de contexte qui m'a été fourni.

«requirement»

Commande tablette

Id = "1.1"

Text = "Réaliser sur tablette l'IHM de commande de direction, de vitesse du kart"

bdd [Modèle] Data [Contexte]

Supprimer la colonne de direction et installer un vérin sur l'axe des roues avants

«block»



«block»
Kart d'origine
references



Commander la direction, la vitesse avec une tablette

«block»



Pour résumer, un kart fonctionnel nous a été donné. Ma tâche consiste à supprimer la colonne de direction sur le kart afin de pouvoir commander la vitesse et la direction du kart grâce à une application sur un appareil mobile.

Solutions retenues

Avec le kart, il nous a fournis : le moteur du kart, sa batterie, un vérin électrique qui remplacera l'actuelle colonne de direction ainsi qu'une carte arduino UNO pour remplacer l'actuel microcontrôleur. À savoir que la batterie alimentera le moteur et le vérin à 24 V.

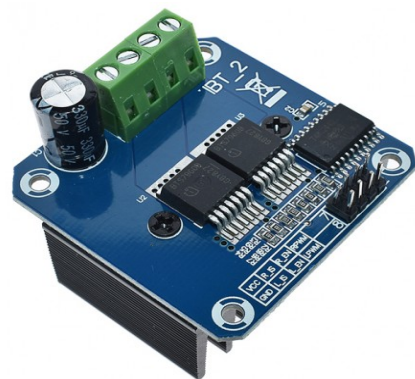
Il faut donc que je trouve une interface de puissance pour le moteur, une interface de puissance pour le vérin électrique et un module permettant une communication sans-fil.

Pour la communication sans-fil, j'ai choisi la communication Bluetooth, car elle est très simple d'utilisation, compatible avec la carte arduino, peu onéreux et à une portée allant jusqu'à 10 m. J'ai choisi le module HC05 : <https://www.gotronic.fr/art-module-bluetooth-hc05-26097.htm> .

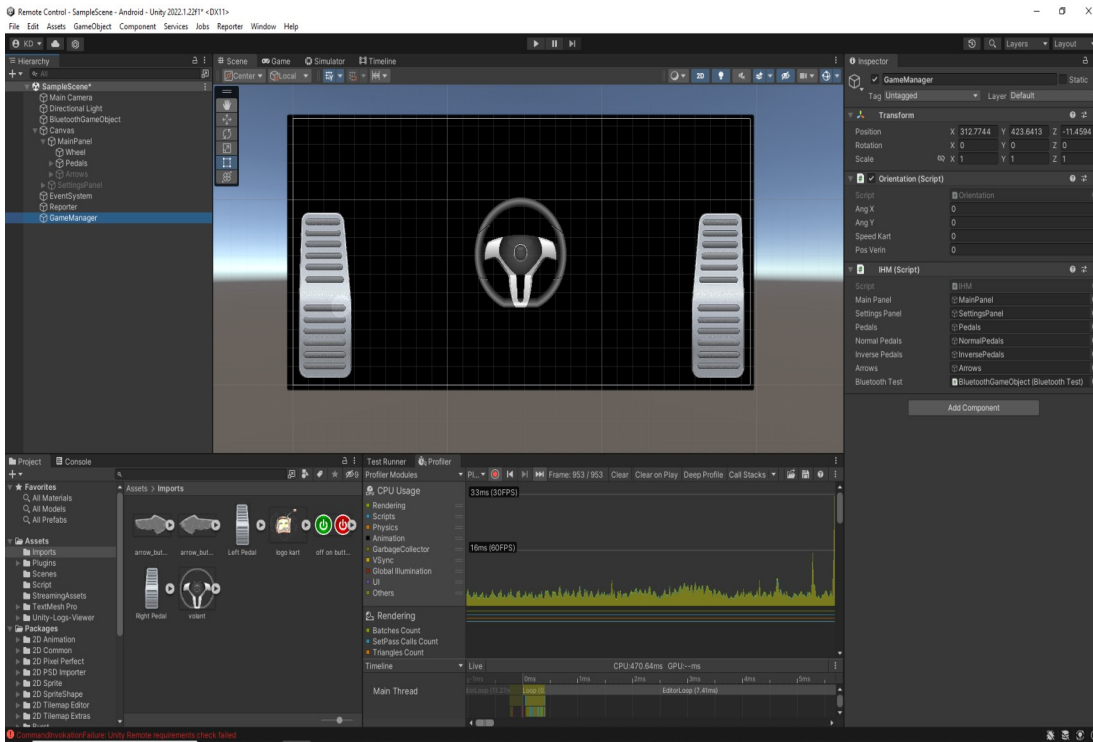


L'interface de puissance pour le vérin électrique est le PKA03 Shield : <https://www.gotronic.fr/art-kit-shield-moteurs-2-x-2-5-a-ka03-20845.htm>, il s'agit d'un shield donc il se branche sur la carte arduino et il a deux sorties à 2,5 A.

L'interface de puissance pour le moteur du kart est le BTS7960 : <https://www.botnroll.com/en/controllers/3589-bts7960-43a-dual-h-bridge-high-power-motor-driver.html>, j'ai choisi cette interface, car il peut supporter jusqu'à 43 A ce qui devrait suffire au fort courants du moteur.



Les deux interfaces de puissances sont des pont en H, car il me faut pouvoir faire varier la tension distribuée.

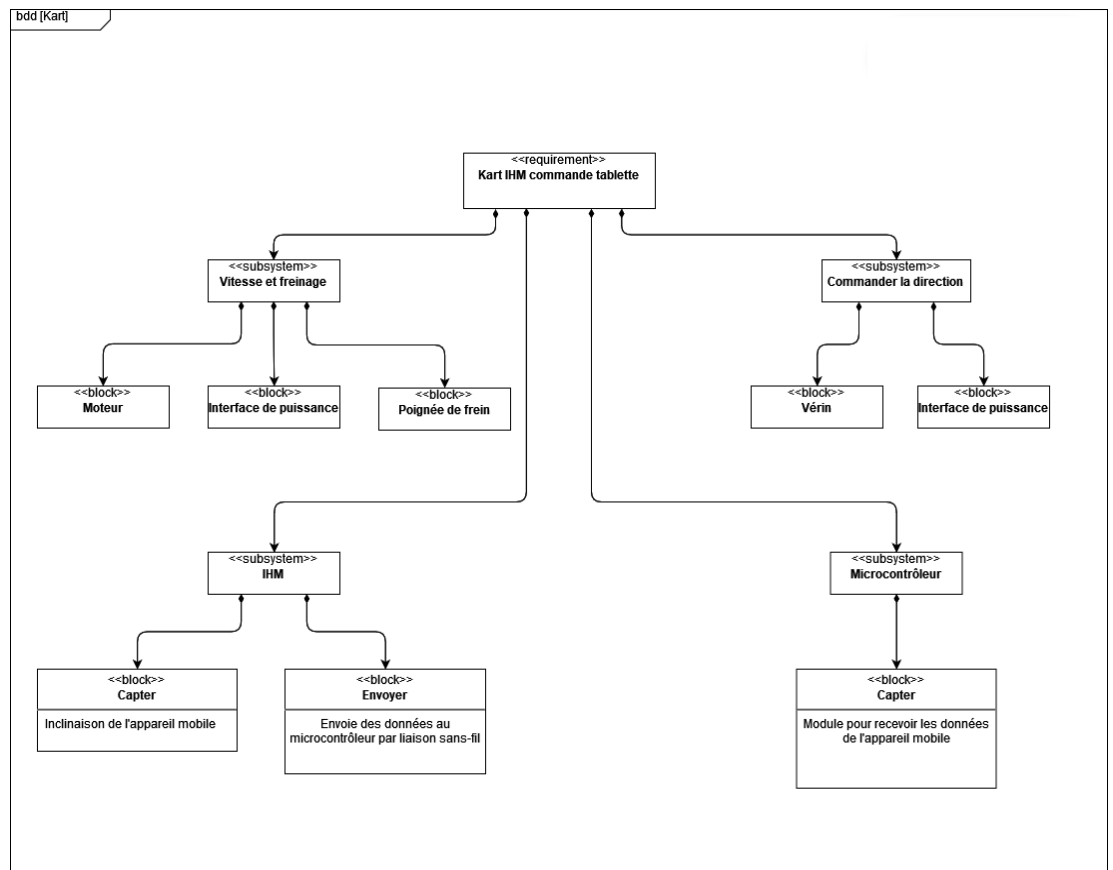


Pour le développement du logiciel sur la tablette, j'ai choisi l'environnement Unity. Il s'agit d'un environnement de jeu vidéo en 2D et 3D. Nous y programmons en C# (semblable au C++). Il serait alors excessif de s'en servir pour développer un petit logiciel sur portable.

Néanmoins, j'ai déjà réalisé plusieurs projets avec ce logiciel et ce langage. De plus, l'autre solution serait d'utiliser l'environnement App Inventor qui est bien plus raisonnable, mais qui utilise le langage de bloc, ce que j'ai bien plus de mal à utiliser. Sachant que le développement sur Unity est un de mes passe-temps favoris, j'ai donc opté sans hésitation pour cet environnement.

Diagramme sysml de définition de blocs

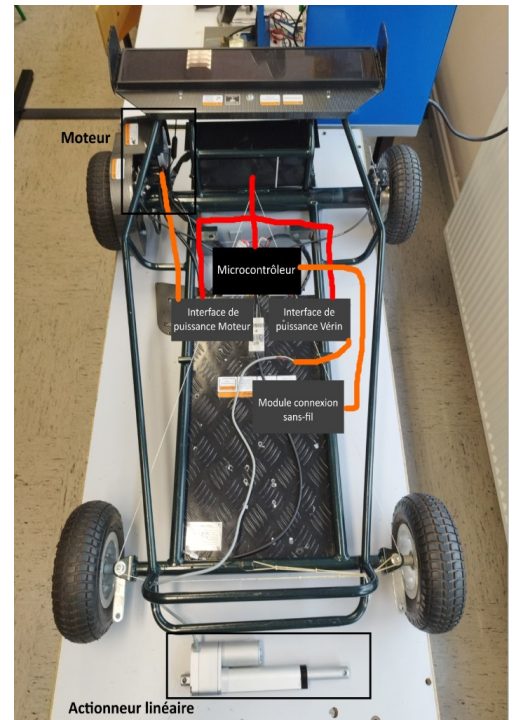
Le diagramme sysml de définition de blocs nous permet dans un premier temps de hiérarchiser le système complet sans donner les caractéristiques précises de chaque bloc.



Caractérisation

Schéma d'ensemble

Sur le schéma d'ensemble, nous pouvons voir le kart sans la colonne de direction et j'y ai ajouté les composants et les liaisons qui seront présente.



Caractéristiques du vérin électrique

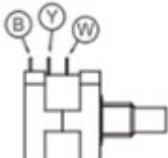
Ce vérin peut s'étendre jusqu'à 100 mm et il a une puissance de poussée de 250 N.

Typical current * (A)			
minimum load		maximum load	
12 Vdc	24 Vdc	12 Vdc	24 Vdc
1.2	0.6	3.2	1.6
1.2	0.6	3.2	1.6
1.2	0.6	3.2	1.6
1.2	0.6	3.2	1.6
1.2	0.6	3.2	1.6



Nous l'alimenterons en 24 Vdc, ce qui correspond à un courant nominal de 1,6 A. Il correspond donc à notre interface de puissance choisie.

La particularité de ce vérin est qu'il est muni d'un potentiomètre. C'est intéressant de pouvoir étendre et rétracter sa longueur, mais si nous ne pouvons pas accéder à la position instantanée alors le vérin devient inutile. Le potentiomètre est une résistance (10 kOhm) qui varie en fonction de la position du vérin, ainsi le microcontrôleur pourra recevoir une tension proportionnelle à la longueur de cours du vérin.

Power		Potentiometer (10 kOhm)			
Red	Black	White	Yellow	Blue	
M+	M-	GND	Vcc	Data	

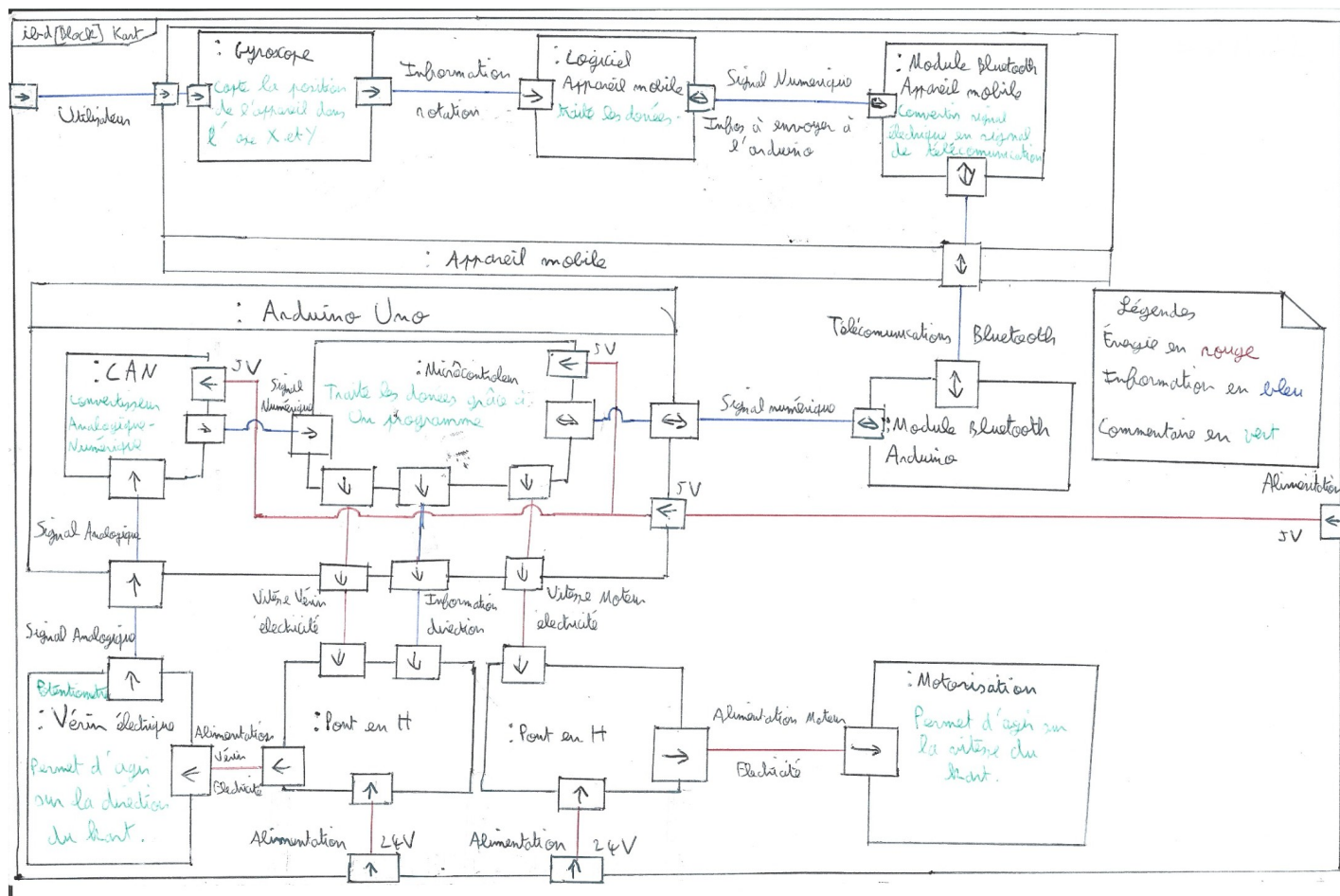
Pour la puissance, il sera relié à l'interface de puissance. Le potentiomètre sera alimenté par le microcontrôleur et lui enverra les données.

Caractéristiques du moteur du kart

Nous avons remarqué grâce à une pince ampèremétrique que le courant du moteur atteint au maximum 17 A. Le pont en H pourra donc supporter son courant. Nous avons aussi décidé que le kart ne fera pas de marche arrière pour simplifier la démarche. L'interface de puissance sera alimentée en 24 V par la batterie et distribuera l'énergie au moteur en fonction du signal de commande provenant du microcontrôleur.

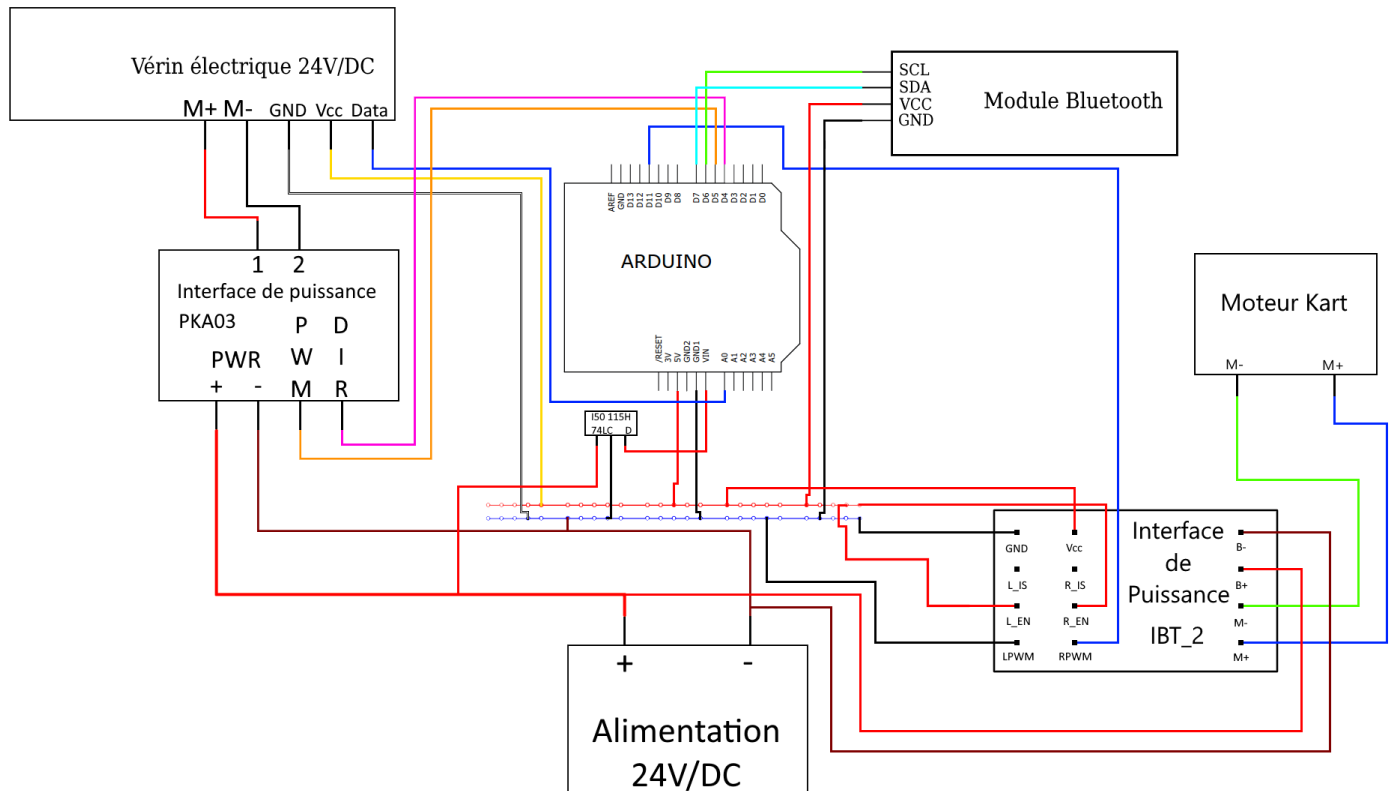
Mise en œuvre

Diagramme sysml de blocs internes



Le diagramme sysml de blocs internes permet de mieux visualiser les flux d'informations et d'énergies.

Schéma électrique

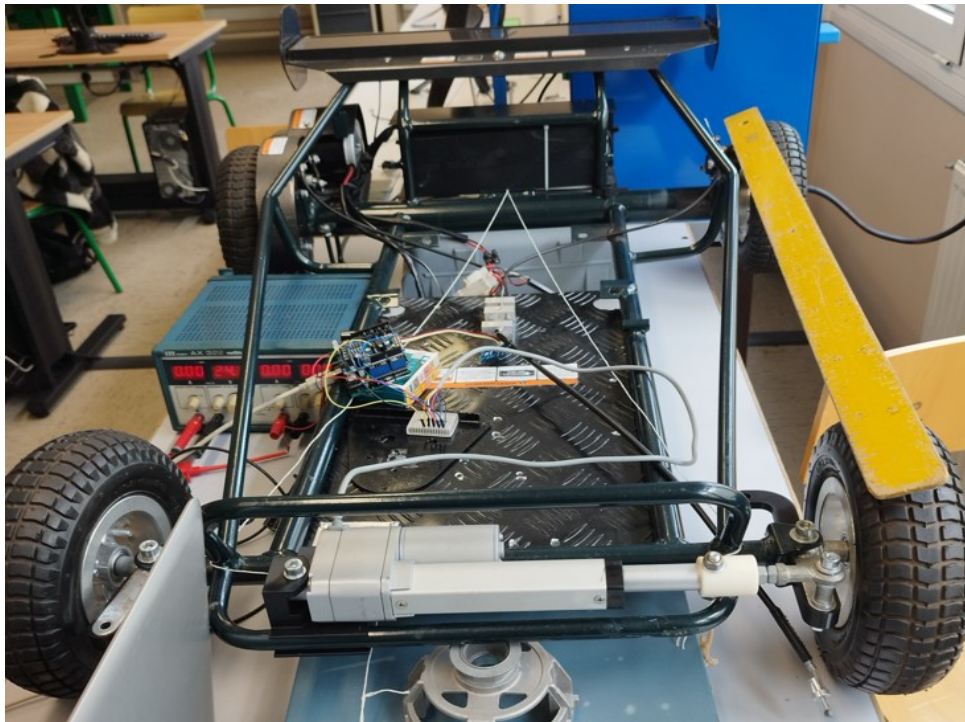


Programme pour le vérin électrique et le moteur du kart

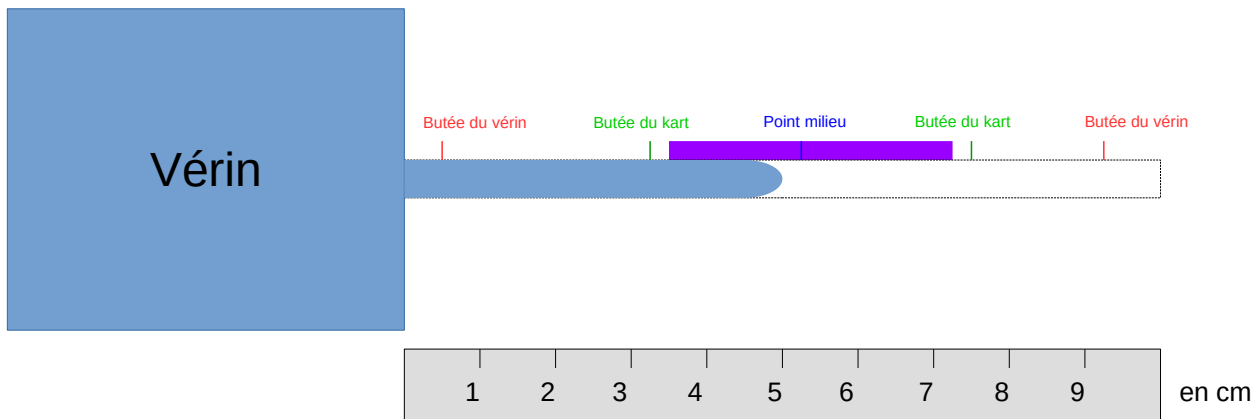
Nous allons commencer avec le vérin, pour rappel, il est doté d'un potentiomètre qui permet de recevoir une tension proportionnelle à la distance parcourue par le vérin. Cette tension sera au maximum de 5 V, donc si le vérin est étendu au maximum (100 mm), la tension du potentiomètre sera de 5 V. Les données du potentiomètre vont être traité par la carte arduino UNO. Ce signal analogique sera d'abord converti en signal numérique sur 10 bits (1024 valeurs) grâce au CAN. Pour faire le lien entre la valeur numérique et la distance en mm voici le tableau :

Distance (en mm)	Valeur numérique (0 à 1023)	Lien mathématique
100	1023	$x = y * 100 / 1023$
x	y	$y = x * 1023 / 100$

Après l'installation du vérin sur le kart et les nombreux réglages sur la roue réalisés, j'ai dû trouver la longueur d'extension du vérin parfaite pour que la roue qui y est relié soit parfaitement parallèle avec les roues situées à l'arrière du kart.



Nous n'utiliserons pas les 100 mm de course du vérin, car il est soumis aux butées du kart, voici un schéma pour mieux comprendre comment nous allons utiliser le vérin :



En **vert**, il s'agit des butées lorsque le vérin est installé sur le kart. Le point milieu est représenté en **bleu**, il est situé à 52,73 mm de course (540 en valeur numérique). J'ai décidé de laisser une marge d'environ 3 mm avant les butées du kart pour éviter les mauvaises surprises. Donc, la course que peut effectuer le vérin sur le kart est représentée en **violet** (soit environ 37 mm).

Le fonctionnement du programme pour le vérin est relativement simple, pour rappel, dans un programme, nous parlons généralement en valeur numérique et non en unité de mesure. Pour convertir une distance en grandeur numérique vers une unité de mesure, referez vous au tableau p6.

The screenshot shows the Arduino IDE serial monitor window titled 'COM9 (Arduino/Genuino Uno)'. It displays the following data in a text area:

```
Potentiometre :321
Sens moteur :1

cible :539
PWM :150
Potentiometre :328
Sens moteur :1

cible :539
PWM :150
Potentiometre :324
Sens moteur :
```

At the bottom of the window, there are controls: a checked 'Défilement automatique' button, a 'Pas de fin de ligne' dropdown, a '9600 baud' dropdown, and an 'Effacer la sortie' button.

Voici un extrait du moniteur série (il s'agit d'une interface qui permet de communiquer directement avec la carte arduino UNO), nous indiquons une valeur « cible » qui correspond à la valeur du potentiomètre que l'on souhaite atteindre.

```
void TrajectoireVerin()
{
    if (posPotentiometre < cibleVerin)    // le verin doit
    {
        differenceValeur = cibleVerin - posPotentiometre; /

        sensVerin = true;
    }

    if (posPotentiometre > cibleVerin)    // le verin doit
    {
        differenceValeur = posPotentiometre - cibleVerin; /

        sensVerin = false;
    }

    if (differenceValeur > 165)            // varie la vites
    {
        avancementVerin = 200;
    } else if (differenceValeur > 100)
    {
        avancementVerin = 100;
    } else if (differenceValeur > 35)
    {
        avancementVerin = 50;
    } else if (differenceValeur > 1)
    {
        avancementVerin = 45;
    } else {
        avancementVerin = 0;
    }

    AjustementPWMVerin();
}

void AjustementPWMVerin()
{
    if (connectedBT)
    {
        analogWrite(pwmVerin, avancementVerin);
        digitalWrite(directionVerin, sensVerin);
    }
}
```

À chaque instant, le programme calcule la différence entre la valeur cible et la valeur reçu par le potentiomètre pour déterminer si le vérin doit s'étendre (sens vérin = 1) ou se rétracter (sens vérin = 0), de plus, plus la cible est éloigné de la position actuelle du vérin, plus la vitesse du moteur sera élevé pour éviter de créer trop de latence entre l'action de l'utilisateur et l'action sur le vérin. Durant les tests, j'y indiquais la valeur cible manuellement, le but est que ce soit l'appareil mobile qui lui envoie automatiquement.

L'interface de puissance envoie une tension au vérin proportionnelle à ce qu'elle reçoit de la carte arduino. Par exemple : si le signal de commande est de 5 V, alors l'interface de puissance distribuera une tension de 24 V au vérin. Ou encore, si le signal de commande est de 2,5 V, alors l'interface distribuera une tension de 12 V au vérin. Comme la tension est moins importante, alors la vitesse du moteur sera aussi moins importante.

Pour faire varier une tension entre 0 et 5 V, nous utilisons la fonction PWM de la carte arduino. Il s'agit d'une valeur sur 8 bits (256 valeurs) qui permet de varier une tension efficace grâce au rapport cyclique. Un PWM de 128 correspond donc à un rapport cyclique de 50% soit une tension efficace de 2,5 V.

Pour le moteur du kart, nous devons faire varier sa vitesse, il nous suffit donc d'envoyer un signal de commande entre 0 et 5 V qui sera proportionnel à la tension distribuée par l'interface de puissance du moteur.

```
void Moteur(int vitesseMoteur)
{
    analogWrite(pwmAvantMoteur, vitesseMoteur);
}
```

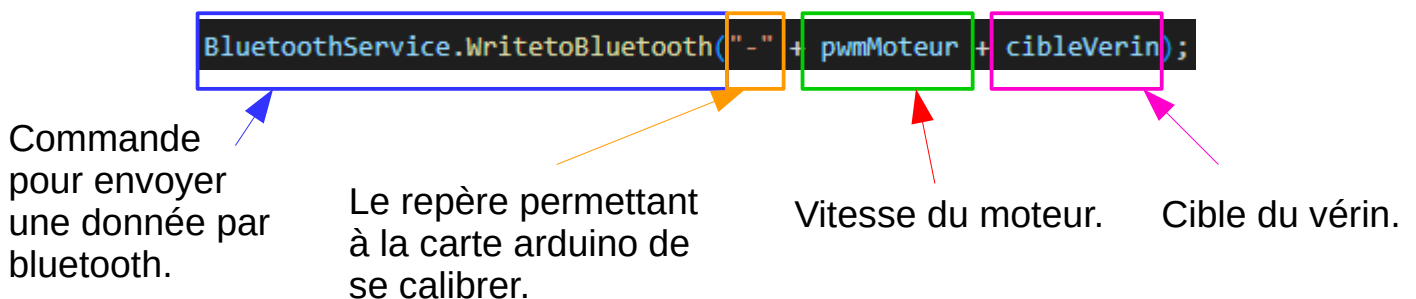
Trame des données envoyés par bluetooth

Finalement, le logiciel sur la tablette doit envoyer deux données :

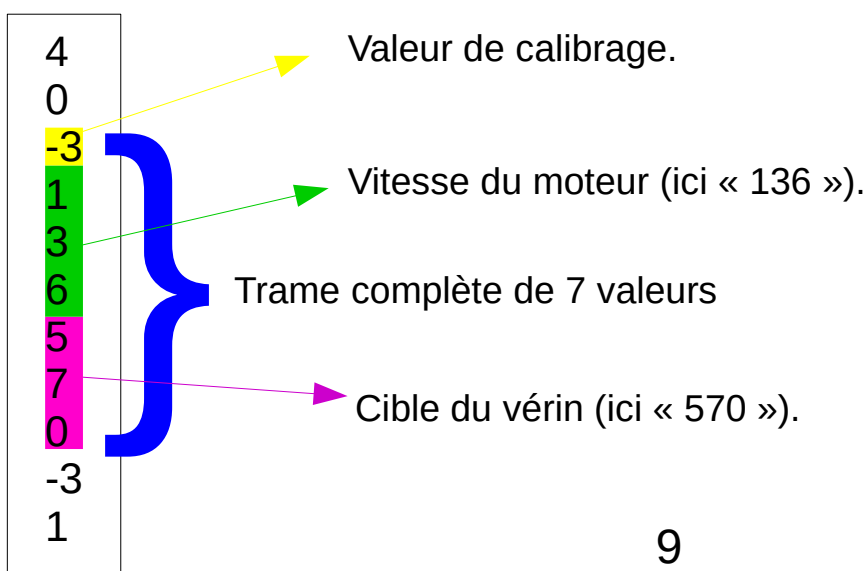
- la vitesse du moteur (entre 0 et 255)
- la valeur cible pour le vérin (entre 0 et 1023)

Remarque importante pour comprendre la suite : les trames sont envoyés en boucle et lorsque le logiciel envoie le caractère : « - » , le module bluetooth connecté à l'arduino recevra la valeur « -3 » .

Voici comment une trame est envoyé depuis le logiciel :



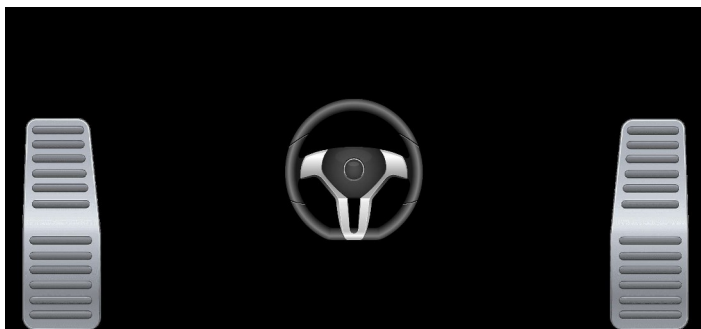
Maintenant, voici ce que reçois la carte arduino UNO :



Pour que le programme de la carte sache s'il s'agit d'une valeur pour la vitesse du moteur ou pour la cible du vérin, j'ai établi une règle simple : avant que la vitesse du moteur ne soit envoyée, il faut envoyer le caractère : « - » . Ainsi, lorsque la carte arduino reçoit la valeur « -3 », alors la prochaine valeur est consacré à la vitesse du moteur, sinon il s'agit de la cible du vérin. J'ai choisi le caractère « - » car il fallait impérativement que ce soit un caractère qui ne puisse jamais être présent dans les deux données envoyés, pour éviter un dysfonctionnement. Or, ces deux données ne seront jamais négatives.

Fonctionnement du logiciel pour la tablette

Ma seule interrogation, lorsque j'ai choisi d'utiliser Unity pour développer le logiciel, était que je m'étais encore jamais servis de cet environnement pour communiquer avec un module bluetooth connecté à un arduino UNO. Après avoir réussi cela, je me suis penché sur l'aspect graphique du logiciel :



J'ai réalisé de nombreux croquis, car je tenais à réaliser une interface soignée et facile d'utilisation, je pense avoir réussi. En cas de préférence ou de problème avec le gyroscope de l'appareil, j'ai prévu différents modes d'utilisation :

- Un mode avec l'utilisation du gyroscope pour la vitesse et la direction.
- Un mode avec l'utilisation du gyroscope pour la direction et des boutons variables pour la vitesse (plus nous touchons le haut de la pédale, plus la vitesse sera forte.)
- Un mode avec l'utilisation du gyroscope pour la vitesse et des boutons avec des flèches pour la direction.

Comme j'utilise Unity, l'utilisation du gyroscope est très simple :

`Gyroscope gyro;` —————> Définis le gyroscope de l'appareil.

`gyro = Input.gyro;`
`gyro.enabled = true;` —————> Initialise le gyroscope de l'appareil.

`angX = Input.acceleration.x - initAngX;`
`angY = Input.acceleration.y - initAngY;` —————> Récupère la valeur du gyroscope.



Le gyroscope ne retourne pas un angle mais un nombre à virgule en 0 et 2.

angX correspond à la valeur du gyroscope lors de la rotation du téléphone vers la droite et la gauche.

angY correspond à la valeur du gyroscope lors de la rotation du téléphone vers l'avant et l'arrière.

InitAngX et initAngY correspondent aux valeurs du gyroscope lors de l'appui sur le bouton « Réinitialiser », ainsi, cela permet de calibrer le gyroscope en fonction des critères de l'utilisateur.

Après de nombreux essais, j'ai décidé que lorsque le gyroscope atteint une valeur de 0,7 ou supérieur sur l'axe X, alors le vérin devras s'étendre/se rétracter au maximum. Pour rappel (toute unité en valeur numérique) :

- La butée du vérin qui s'étend est de 730.
- La butée du vérin qui se rétracte est de 350.
- Le point milieu du vérin est à 540.
- La distance entre le point milieu et une des butées est de 190.

```
nVerin = Mathf.Round((Mathf.Abs(angX) * 190f) / 0.7f);  
  
if (angX < 0)  
{  
    nVerin = 540 + nVerin;  
}  
else {  
    nVerin = 540 - nVerin;  
}
```

*Mathf.Round() = arrondi
Mathf.Abs() = valeur absolue*

Tout d'abord, le logiciel calcul la distance que devra parcourir le vérin à partir du point milieu. Ensuite, si la valeur retournée par le gyroscope est négative, alors il additionne la distance calculée et la valeur du point milieu, sinon il soustrait la valeur du point milieu par la distance calculé. Ainsi, il obtient la cible du vérin.

Pour la vitesse, j'ai choisi que lorsque le gyroscope atteint une valeur de 0,6 ou supérieur sur l'axe Y, alors le PWM serait au maximum (255).

```
pwmMoteur = Mathf.Round((angY - 0.05f) * 255f) / 0.55f;
```

Dans le programme, nous faisons une soustraction sur les valeurs retournées par le gyroscope et la valeur max (0,6) de 0,05 pour renforcer la sécurité.

Problèmes rencontrés

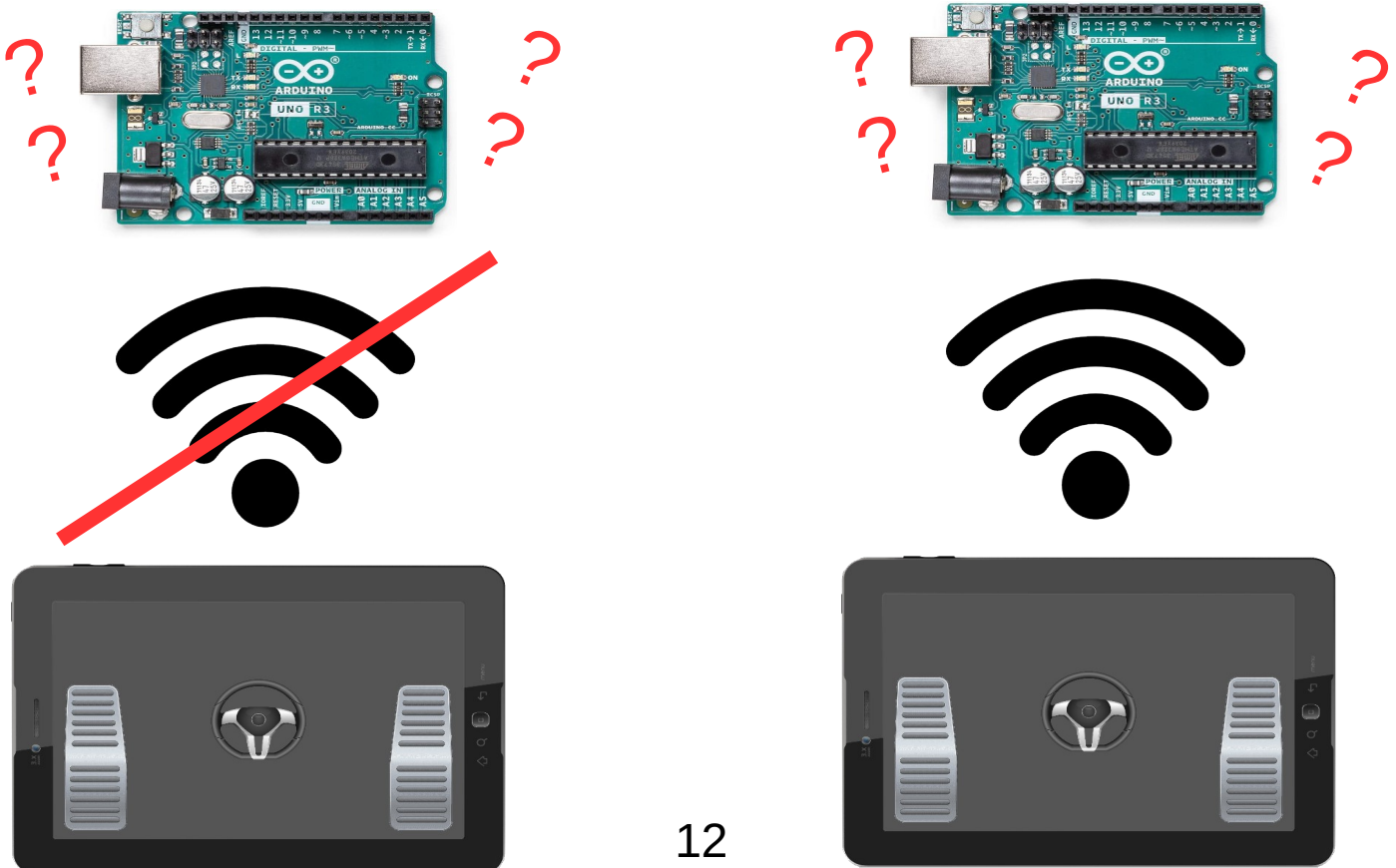
Direction imprécise

Après l'installation du vérin sur le kart, nous avons fait de nombreux réglages pour trouver le point milieu. Pour rappel, il s'agit du point où les roues avant sont parfaitement alignées avec les roues arrière. Pour réaliser ces tests, je devais contrôler le vérin à 0,01 mm près. Cependant, la vitesse du moteur du vérin était trop élevée donc la tige du vérin dépassait sa cible et devait se recalibrer. De plus, la tige s'arrêtait aléatoirement à plus ou moins 1 mm de sa cible.

J'ai donc dû revoir mon programme. Pour régler le problème, j'ai dû décrémer progressivement la vitesse du moteur lorsque la tige du vérin se rapprochait de sa cible. Cette fois-ci, lorsque la tige n'était plus qu'à quelques millimètres de sa cible, la vitesse du moteur devait être très faible. Grâce à cette modification, je pouvais contrôler le vérin avec une précision de 0,01 mm.

Connexion Bluetooth

J'ai rencontré deux problèmes liés à l'échange de données par bluetooth. Dans un premier temps, lorsque la tablette se connecte au module bluetooth de la carte arduino, il est impossible pour la carte de le savoir.



Pour régler ce problème, j'ai décidé d'en faire d'une pierre deux coups et de renforcer la sécurité du système. J'ai donc établi un mot de passe que le logiciel sur la tablette enverra par bluetooth pour donner l'accès à la carte seulement à mon logiciel et pour prévenir la carte de la mise en marche du système. Lors de la déconnexion bluetooth de la tablette, il enverra à nouveau le mot de passe pour arrêter la carte arduino. Pour mieux comprendre les programmes, le mot de passe est : 973553312. Voici le code sur la tablette qui s'exécute lorsque le bouton pour se connecter à la carte est pressé :

```
public static bool StartBluetoothConnection(String DeviceName)
{
    if (Application.platform == RuntimePlatform.Android)
    {
        string connectionStatus = "non";
        try
        {
            connectionStatus = BluetoothConnector.Call<string>("StartBluetoothConnection", DeviceName);
            BluetoothConnector.Call("PrintOnScreen", context, connectionStatus);
            if (connectionStatus == "Connected")
            {
                WritetoBluetooth("973553312");
                return true;
            } else
            {
                return false;
            }
        }
        catch (Exception e)
        {
            BluetoothConnector.Call("PrintOnScreen", context, connectionStatus);
            return false;
        }
    }
    else return false;
}
```

Vérifie que le module bluetooth de la carte arduino est appairé à la tablette.

Si oui

Envoie le mot de passe.

Maintenant, voici comment la carte arduino donne l'accès à la tablette. D'abord, je déclare une liste avec tous les chiffres constituant le mot de passe.

```
int motdepasse[] = {9, 7, 3, 5, 5, 3, 3, 1, 2};
```

Puis, après qu'une valeur soit reçue sur le module bluetooth, regardera s'il correspond au mot de passe :

```
recvChar = blueToothSerial.read();
```

Enregistre la valeur reçu depuis le bluetooth dans la variable recvChar.

```
recvChar = recvChar - 48;
```

```
if (motdepasse[i] == recvChar)
{
    i = i + 1;
} else {
    i = 0;
}
```

Permet de convertir un nombre de la table ASCII vers un nombre entier (d'où « - » devient « -3 ») .

```
if (i == 9)
```

Test la correspondance de la valeur reçu dans l'ordre du mot de passe.

```
connectedBT = !connectedBT;
centaine = 5;
dixaine = 4;
i = 0;
```

Si le mot de passe entier a été reçu, alors on alterne allumé/éteint.

```
if (!connectedBT)
{
    capteData = false;
}
```

Réinitialise le programme si le bluetooth a été déconnecté.

J'ai eu un second problème concernant la connexion bluetooth. Pour une raison que j'ignore, le module bluetooth de la carte arduino reçoit les données : caractère après caractère. Imaginons que le logiciel envoie le mot de passe : 973553312, la carte recevra : 9 ; 7 ; 3 ; 5 ; 5 ; 3 ; 3 ; 1 ; 2. Pour le mot de passe, ce n'est pas gênant. Cependant, pour les données concernant le vérin et le moteur, la carte doit les regrouper par 3. Prenons comme exemple la valeur 540 destinée au vérin :

```
if (centaine == -1 && capteData && recvChar != -3)
{
    centaine = recvChar * 100;
} else if (dixaine == -1 && capteData && recvChar != -3)
{
    dixaine = recvChar * 10;
} else if (capteData && recvChar != -3)
{
    valeur = centaine + dixaine + recvChar;
    centaine = -1;
    dixaine = -1;
```

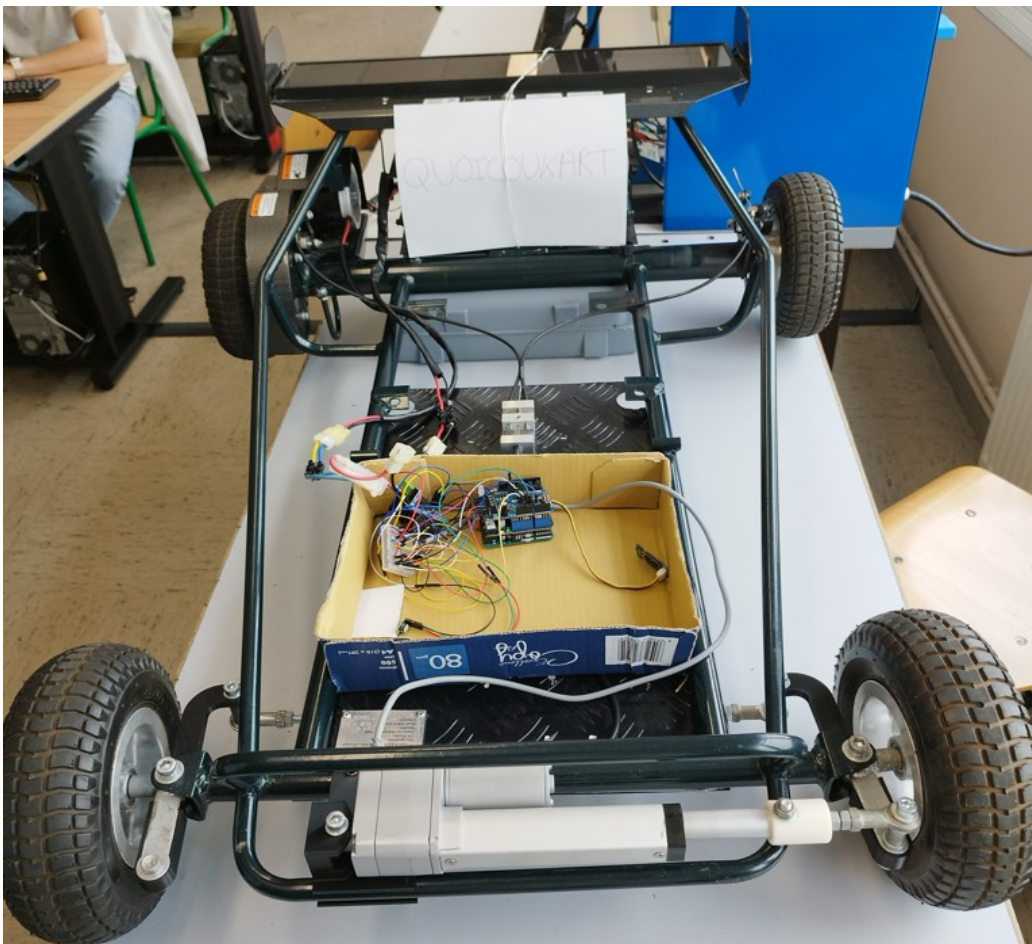
La première valeur reçue est « 5 » que l'on multiplie par 100 .

La seconde valeur reçue est « 4 » que l'on multiplie par 10 .

La valeur regroupée = 500 + 40 + 0 (la dernière valeur reçue) .

Se réinitialise pour regrouper 3 nouvelles valeurs.

Intégration



L'intégration des programmes a été très simple, car j'avais déjà anticipé les noms de toutes les variables ainsi que toutes les broches. Pour le matériel, j'ai tout restreint dans une boîte en carton pour éviter tout court-circuit avec le support métallique du kart.

Conformité au cahier des charges

J'ai aménagé le kart qui nous a été donné, grâce aux pièces fabriquées par Théo et Adrien, pour que la direction et la vitesse du kart puissent être contrôlé à distance par l'utilisateur grâce à une application sur tablette. Le résultat du projet est donc conforme au cahier des charges.

Conclusion personnelle

Pour conclure, nous avons fait face à de nombreuses difficultés et de nombreux questionnements, mais j'ai réussi tous les résoudre dans l'ensemble. Je suis satisfait du résultat, car en plus d'être conforme au cahier des charges, il est plutôt qualitatif. C'est assez rare pour un projet comme celui-ci que le résultat soit aussi concluant, il comporte souvent des imperfections. Pourtant, après avoir fait essayé le kart pendant une heure, que ce soit par des personnes de mon groupe ou externe au projet, il n'y a aucune latence gênante entre les envies de l'utilisateur et l'exécution sur le kart, la direction et la vitesse sont précises et le kart est dans son entièreté intuitive à utiliser.

Je pense m'être très bien organisé et j'ai donc réussi à finir le projet dans le temps malgré les problèmes rencontrés.

Cependant, si j'aurais eu plus de temps, j'aurais voulu ajouter la possibilité de freiner, malgré le fait que le kart ne roule pas vite, cette fonctionnalité aurait pu être intéressante à mettre en place. Cela aurait aussi pu être intéressant d'y ajouter des capteurs de distance pour éviter les obstacles. Ou encore, ajouter sur l'IHM une carte avec un relevé de tout le chemin parcouru par le kart.

Je remercie :

Théo Allaire et Adrien Chauvel pour leurs travaux sur le kart sans qui le kart n'aurait jamais pu fonctionner de tel sortes, ainsi que leurs conseils.

Akram Lakhil pour les fonctionnalités ajoutés.

Mr Leriche pour les précieux conseils sur les énergies et les puissances.

Mr Barbé pour les nombreux réglages qui ont permis au kart d'être aussi réussi, ainsi que ses conseils instructifs.

Mr David de m'avoir choisi pour ce projet auquel je me souviendrai comme l'un de mes projets les plus amusants à réaliser, l'aide qu'il m'a apporté pour des moments du projet cruciaux, ainsi que sa confiance lors de mes différents tests et pour me laisser utiliser Unity.